

LoRa P2P TX Application

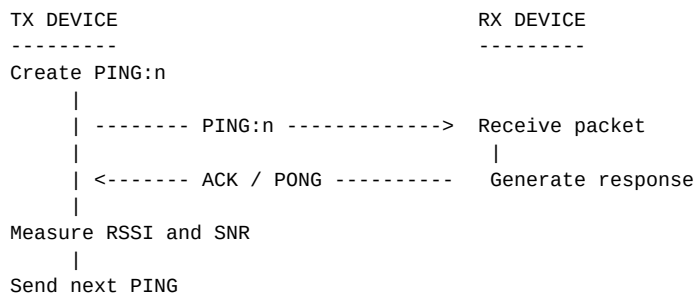
Detailed Technical Explanation of subghz_phy_app.c

Purpose: This document explains the STM32 Sub-GHz / LoRa Point-to-Point transmitter application, its state machine, callbacks, packet flow, connection detection, RSSI/SNR monitoring, timeout handling, and recommended improvements.

Application behavior: The transmitter periodically sends a PING packet, switches to receive mode, waits for an ACK or PONG response, evaluates the radio link, and automatically retries when communication fails.

1. System Overview

The application implements a simple application-level heartbeat protocol over a LoRa Point-to-Point (P2P) radio link. It does not use the LoRaWAN network stack. Instead, two devices communicate directly using matching LoRa radio parameters.



2. Main Communication Cycle

Step	Action
1	Initialize STM32 Sub-GHz radio and configure LoRa parameters.
2	Create a packet such as PING:1.
3	Transmit the packet using Radio.Send().
4	OnTxDone callback reports successful transmission.
5	Application switches the radio into receive mode.
6	Wait for ACK or PONG from the remote device.
7	If received, store RSSI and SNR and mark the link CONNECTED.
8	Increase the packet sequence number and send the next PING after 1500 ms.
9	If timeout/error occurs, mark the link DISCONNECTED and retry after 500 ms.

3. Included Header Files

Header	Purpose
platform.h / sys_app.h	STM32 platform and system application support.
subghz_phy_app.h	Application configuration and LoRa parameter definitions.

radio.h	Radio driver interface including Init, Send, Rx, and configuration APIs.
stm32_seq.h	STM32 task sequencer used to defer processing outside callbacks.
timer.h	Timer-related middleware support.
string.h	Provides memcpy(), strlen(), and strcmp().
stdio.h	Provides sprintf() for packet and UART message formatting.

4. Application State Machine

The enum AppStates_t represents the most recent radio event. The callback functions update State and schedule SubghzApp_Process(), which performs the main application work.

```
typedef enum
{
    APP_IDLE,
    APP_TX,
    APP_TX_TIMEOUT,
    APP_RX,
    APP_RX_TIMEOUT,
    APP_RX_ERROR,
} AppStates_t;
```

State	Meaning
APP_IDLE	No pending application action.
APP_TX	Transmission completed successfully.
APP_TX_TIMEOUT	Transmission timed out or failed.
APP_RX	A packet was received successfully.
APP_RX_TIMEOUT	No response arrived before the receive timeout.
APP_RX_ERROR	A receive error occurred.

5. Connection State

The application uses a second state variable to represent whether the remote node is responding.

```
typedef enum
{
    CONN_DISCONNECTED,
    CONN_CONNECTED
} ConnState_t;
```

```
static ConnState_t LinkState = CONN_DISCONNECTED;
```

This is different from the radio event state. For example, APP_RX means a packet was received, while CONN_CONNECTED means the application has received a valid ACK/PONG and considers the remote peer alive.

6. Important Variables

Variable	Purpose
PktSeqNum	Packet sequence number. Starts at 1 and increments after successful ACK/PONG reception.
State	Current application/radio event state.
Buffer	Stores transmitted and received packet data.

BufferSize	Stores the current packet length.
RssiValue	RSSI of the most recently received packet.
SnrValue	SNR of the most recently received packet.
LinkState	Logical connection status of the remote node.
huart1	UART used for debug and status messages.

7. Initialization: SubghzApp_Init()

The initialization function performs three major tasks: registers the application task with the STM32 sequencer, registers radio callbacks, and configures the LoRa radio.

```
UTIL_SEQ_RegTask(  
    SUBGHZ_PHY_PROCESS_TASK,  
    UTIL_SEQ_RFU,  
    SubghzApp_Process  
);
```

This registers SubghzApp_Process() as a scheduled task. Radio callbacks remain short and simply set a state and request this task to execute.

8. Radio Event Callback Registration

```
RadioEvents.TxDone      = OnTxDone;  
RadioEvents.RxDone      = OnRxDone;  
RadioEvents.TxTimeout   = OnTxTimeout;  
RadioEvents.RxTimeout   = OnRxTimeout;  
RadioEvents.RxError     = OnRxError;  
  
Radio.Init(&RadioEvents);
```

The radio middleware invokes these functions when the corresponding hardware events occur.

9. Radio Configuration

```
Radio.SetChannel(RF_FREQUENCY);
```

Sets the RF carrier frequency. Both ends of a P2P link must use compatible radio settings.

The transmitter is configured with parameters including TX output power, LoRa bandwidth, spreading factor, coding rate, preamble length, CRC, IQ inversion, and timeout.

The receiver is configured with corresponding parameters so that the incoming LoRa signal can be decoded.

Parameter	Effect on Link
Frequency	Both nodes must use the same channel/frequency plan.
Bandwidth (BW)	Higher bandwidth gives faster transmission but generally reduces sensitivity.
Spreading Factor (SF)	Higher SF improves sensitivity/range but increases airtime and reduces data rate.
Coding Rate (CR)	Adds forward error correction; stronger protection generally increases airtime.
TX Power	Higher power can improve link margin but is subject to hardware and regulatory limits.
Preamble Length	Helps receiver synchronization; longer preamble increases airtime.
CRC	Detects corrupted packets.
IQ Inversion	Must be compatible between communicating devices.

10. SendPing() Function

```
static void SendPing(void)  
{  
    snprintf((char *)Buffer,  
        BUFFER_SIZE,  
        "PING:%lu",  
        (unsigned long)PktSeqNum);
```

```

    BufferSize = (uint16_t)strlen((char *)Buffer);

    Radio.Send(Buffer, BufferSize);
}

```

If PktSeqNum is 5, the transmitted application payload is PING:5. The packet is then passed to the radio driver through Radio.Send().

11. Transmission Complete Callback

```

static void OnTxDone(void)
{
    State = APP_TX;

    UTIL_SEQ_SetTask(
        SUBGHZ_PHY_PROCESS_TASK,
        UTIL_SEQ_RFU
    );
}

```

The callback does not perform the complete communication logic. It records APP_TX and schedules the main processing task. This keeps the event callback short.

12. Switching from TX to RX

When SubghzApp_Process() sees APP_TX, it prints a status message and immediately starts the receiver.

```

case APP_TX:
    // Print packet status
    Radio.Rx(RX_TIMEOUT_VALUE);
    break;

```

The intended protocol is therefore half-duplex: transmit a PING, then listen for the remote device response.

13. Receive Callback and Signal Measurements

```

BufferSize = size > BUFFER_SIZE ? BUFFER_SIZE : size;
memcpy(Buffer, payload, BufferSize);

RssiValue = rssi;
SnrValue = LoraSnr_FskCfo;

State = APP_RX;

```

The received packet is copied into Buffer and its radio measurements are stored. The buffer length is limited to BUFFER_SIZE to reduce overflow risk.

14. ACK/PONG Validation

The application treats a received packet as successful if it starts with ACK or PONG.

```

strncmp((char *)Buffer, "ACK", 3) == 0 ||
strncmp((char *)Buffer, "PONG", 4) == 0

```

This means ACK, ACK:10, PONG, and PONG:10 are accepted. The first successful response changes the logical connection state from DISCONNECTED to CONNECTED.

15. RSSI and SNR

RSSI (Received Signal Strength Indicator) describes the approximate received signal power. Values are normally expressed in dBm, where values closer to zero represent stronger received power.

SNR (Signal-to-Noise Ratio) describes how far the useful signal is above or below the noise floor. LoRa is designed to operate at relatively low and even negative SNR values, depending on spreading factor and other conditions.

RSSI and SNR should be interpreted together. A strong RSSI with poor SNR may indicate significant interference or noise. A weaker RSSI with an acceptable SNR may still produce reliable LoRa decoding.

16. Successful Packet Processing

```
if (valid ACK or PONG)
{
    if (LinkState == CONN_DISCONNECTED)
    {
        LinkState = CONN_CONNECTED;
    }

    // Print RSSI and SNR
    PktSeqNum++;

    HAL_Delay(PING_TX_PERIOD_MS);
    SendPing();
}
```

After a successful response, the packet number increases. The next heartbeat is sent after PING_TX_PERIOD_MS, currently configured as 1500 milliseconds.

17. Timeout and Error Handling

APP_RX_TIMEOUT, APP_RX_ERROR, and APP_TX_TIMEOUT are processed using the same recovery logic.

```
if (LinkState == CONN_CONNECTED)
{
    LinkState = CONN_DISCONNECTED;
}

// Print timeout message
HAL_Delay(PING_RETRY_PERIOD_MS);
SendPing();
```

The sequence number is not increased after a failure. Therefore, the same packet number is retried until a successful response is received.

18. Example Runtime Log

```
--- TX Init Complete: Ready to Connect ---

[TX] Sent Pkt #1, waiting for ACK...

>>> [LINK CONNECTED] Handshake success with RX! <<<

[TX] Received PONG:1 | RSSI=-78 dBm | SNR=8 dB

[TX] Sent Pkt #2, waiting for ACK...
[TX] Received PONG:2 | RSSI=-82 dBm | SNR=5 dB
```

19. Connection Loss Example

```
[TX] Sent Pkt #10, waiting for ACK...

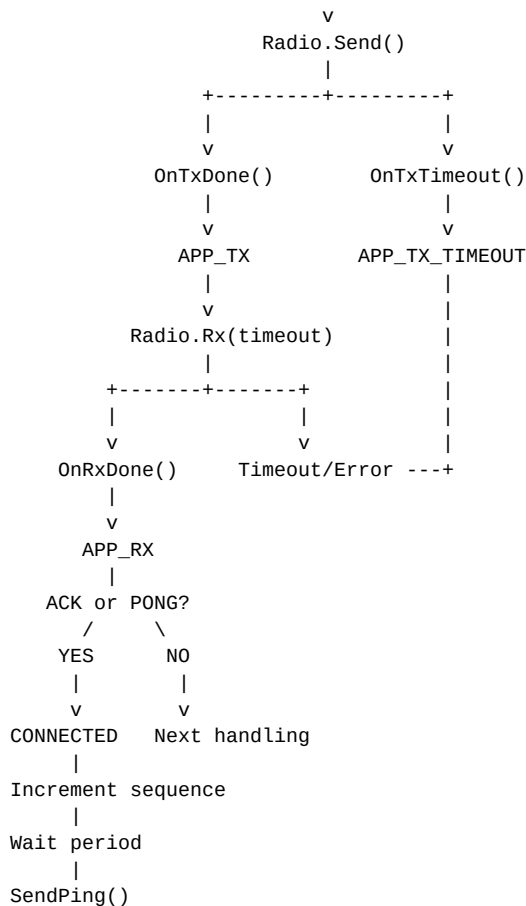
>>> [LINK DISCONNECTED / RECONNECTING...] <<<

[TX] No ACK for Pkt #10 (timeout). Retrying...

[TX] Sent Pkt #10, waiting for ACK...
```

20. Complete Logical State Flow

```
INITIALIZATION
|
v
SendPing()
|
```



21. Recommended Improvements

Improvement	Reason
Replace HAL_Delay()	HAL_Delay() blocks CPU execution. A non-blocking timer or UTIL_TIMER is better for multitasking.
Add retry limit	Unlimited retries may prevent the application from performing other recovery actions.
Validate sequence number	Respond with PONG:n and verify that n matches the transmitted packet.
Add packet type/version	A structured packet format improves future compatibility.
Track packet loss	Count successful and failed attempts to calculate link reliability.
Track moving RSSI/SNR averages	Useful for detecting gradual degradation rather than reacting to a single packet.
Use explicit state reset logic	Makes state transitions easier to maintain as the application grows.
Consider asynchronous timers	Avoids blocking the sequencer while waiting between PING packets.

22. Suggested Improved Packet Protocol

A more reliable application-level protocol could use the packet sequence number in both directions.

```

TX -> RX:  PING:25
RX -> TX:  PONG:25

```

```

TX verifies:
Received sequence == Expected sequence

```

This helps prevent an old delayed packet from being accepted as the response to a new transmission.

23. Key Takeaways

1. This application is a LoRa P2P heartbeat/link-monitoring system.
2. A PING packet is transmitted and the device then listens for ACK or PONG.
3. Successful reception provides RSSI and SNR measurements.
4. A valid response changes the logical state to `CONNECTED`.
5. A timeout or receive error changes the logical state to `DISCONNECTED`.
6. Successful packets advance the sequence number; failed packets retry the same sequence number.
7. The code uses callbacks plus the STM32 sequencer to separate radio events from application processing.
8. The most important future improvement is replacing blocking `HAL_Delay()` calls with asynchronous timers.

24. Final Summary

The code provides a solid basic framework for direct LoRa Point-to-Point communication. It combines packet transmission, response validation, connection detection, automatic retry, and physical link measurements. Functionally, it behaves like a heartbeat protocol: the transmitter continuously checks whether the remote node is alive and responsive.

The next stage of development should focus on a stronger packet protocol, retry limits, non-blocking timing, packet-loss statistics, and long-term RSSI/SNR monitoring. These additions would make the system more suitable for a robust self-healing or long-distance LoRa communication network.